

Final Report

Trey Wiedmann

1 Problem

We are iteratively computing the solution for the 3D Poisson equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = f(x, y, z)$$

f is known, and we solve for u on an N by M by K grid of points inside an arbitrary rectangular boundary, for which the solution on the boundary is known.

2 Implementation

2.1 Distributed memory parallelism with MPI

We initialize ranks in a 3D Cartesian mesh with MPI. The number of ranks in each dimension is configurable. The grid is divided evenly along the ranks in each dimension (though some ranks may have different size due to remainders).

Each rank computes its local update in parallel, and then all ranks perform a halo exchange. Each rank makes a non-blocking receive call to each of its neighbors, and then makes a non-blocking send call to each of them. Finally, each rank waits for all its requests to finish before continuing execution.

To compute the RMS error of the entire solution, each rank computes its local residual sum, and then MPI reduce is called to compute the global RMS error. This is used check for convergence of the solution. Once converged, rank 0 broadcasts a stop signal to all ranks.

Once converged, we write the solution to an output file. MPI file functionalities are used to allow ranks to write their local solutions to the file independently. This is also used to save intermediate solutions if desired. We wrote a visualization tool to display these solutions, and animate how they converge over the lifetime of the solver. These videos are included in the presentation.

2.2 Shared memory parallelism on GPU

We implement a red-black Gauss-Seidel update on the GPU with HIP.

For computation of the RMS error, we also implement a kernel that computes the residual for each point on the local grid and then reduces the sum in shared memory.

For halo exchange, a GPU kernel is used to pack data from the local grid into send buffers.

3 Arithmetic Intensity

3.1 Red-black Update

We read and write u once. We also precompute the values of f on the grid, which we read. If b is the size of our data type in bytes, then this is

$$3NMKb \text{ bytes}$$

Each point on the grid requires 10 floating operations to compute the update, for a total of:

$$10NMK \text{ FLOPs}$$

For an 8 byte data type, our arithmetic intensity is:

$$\frac{10NMK \text{ FLOPs}}{3NMKb \text{ bytes}} = \frac{5}{12} \text{ FLOPs per byte}$$

3.2 Residual Calculation

We read u and the precomputed f once:

$$2NMKb \text{ bytes}$$

We compute the square residual for each point on the grid, amounting to 13 FLOPs per point. Then, we sum all NMK values:

$$13NMK + NMK = 14NMK \text{ FLOPs}$$

Thus our arithmetic intensity is:

$$\frac{14NMK \text{ FLOPs}}{2NMKb \text{ bytes}} = \frac{7}{8} \text{ FLOPs per byte}$$

3.3 MSE Calculation

We read u once:

$$NMKb \text{ bytes}$$

We compute the true solution for each point on the grid, which depends on the equation. Call this U FLOPs. Then, for each grid point, we subtract from our calculated solution and square. Finally, we sum all NMK values.

$$UNMK + 2NMK + NMK = (U + 3)NMK \text{ FLOPs}$$

Thus our arithmetic intensity is:

$$\frac{(U + 3)NMK \text{ FLOPs}}{NMKb \text{ bytes}}$$

4 Experimental Results

4.1 Hardware

We ran all experiments on the AMD HPC fund cloud. Experiments were run on nodes with the MI210 accelerator. Using the peak FP64 performance and memory bandwidth given here, each GPU has a peak performance of:

$$22.6 \text{ TFLOP/s}$$

and a memory bandwidth of:

$$1.6 \text{ TB/s}$$

This is an arithmetic intensity of:

$$\frac{22.6 \text{ TFLOP/s}}{1.6 \text{ TB/s}} = 14.125 \text{ FLOP/byte}$$

We were able to get 8 nodes with GPUs for our experiments, so we multiply given peak performance and memory bandwidth by 8 for analysis.

4.2 Residual and Error vs Iterations

Figure 1 plots the RMS residual and MSE of the solver for one example. These both display a steady exponential decay trend after an initial period of rapid convergence. The MSE eventually levels off.

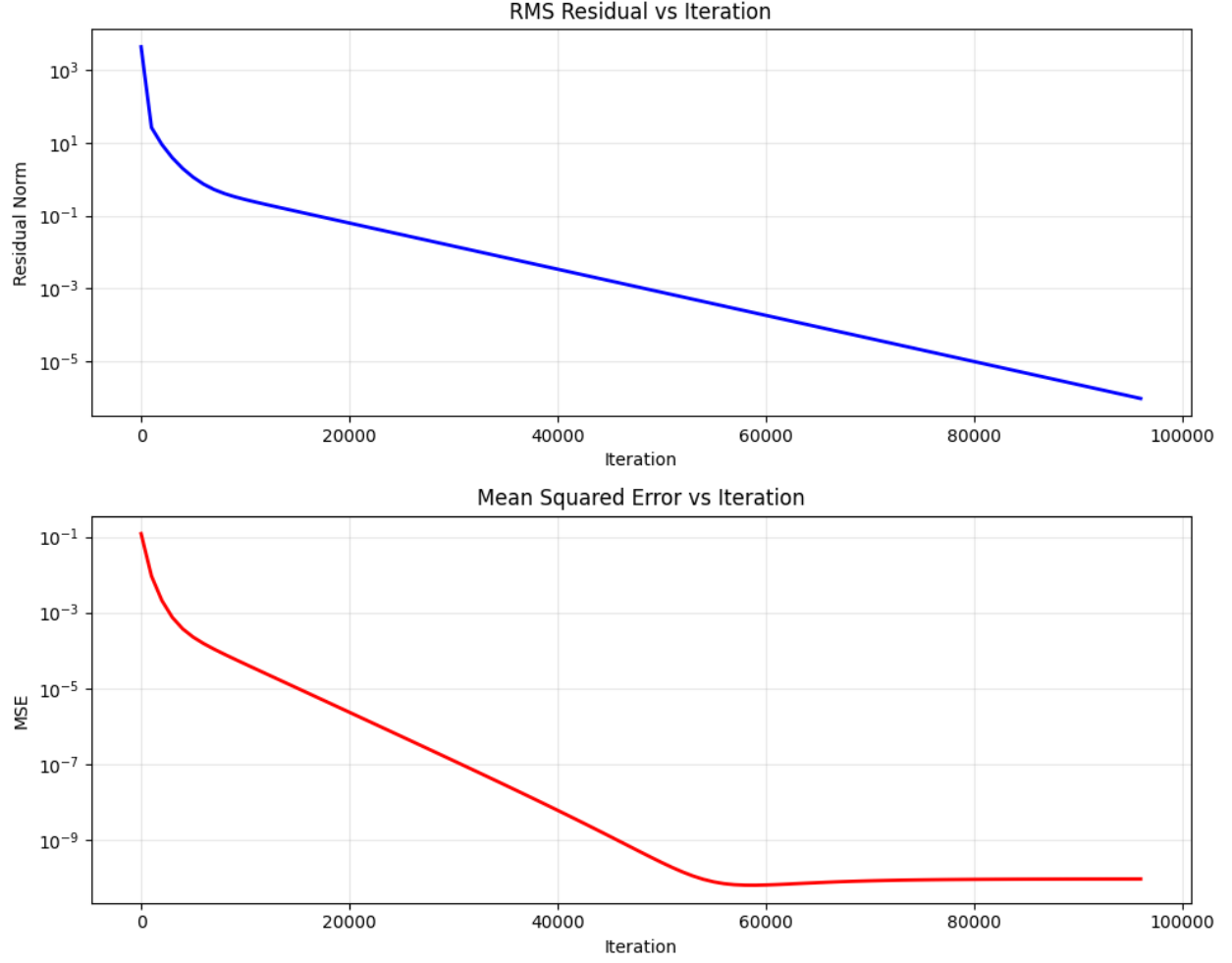


Figure 1: 256x256x256 grid, $f(x, y, z) = \cos(2\pi x) \cos(2\pi y) \cos(2\pi z)$, $x, y, z \in (0, 1)$

4.3 GPU-Aware MPI Communication

We compared the solver with and without using GPU-aware MPI to directly communicate the halo exchange between GPUs. For a 256x256x256 grid with 2x2x2 MPI ranks, a single halo exchange ran 3x faster with GPU-aware communication. Overall execution ran 1.8x faster. For a 512x512x512 grid with 4x4x4 MPI ranks, overall execution ran 2.2x faster. This benefit is made clear when we profile the GPU.

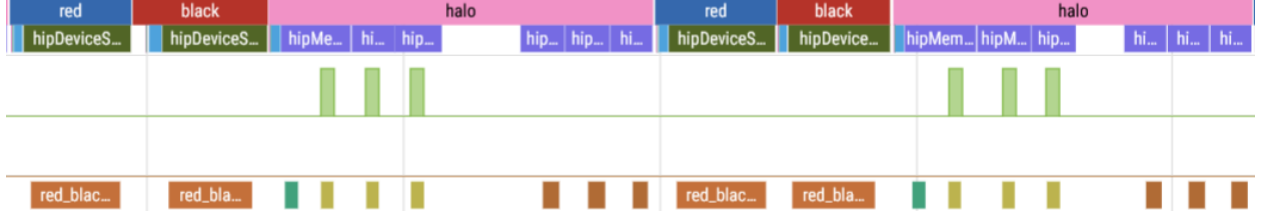


Figure 2: Without GPU-aware. For a node with 3 neighbors, 3 device-to-host copies plus MPI communication plus 3 host-to-device copies are needed.

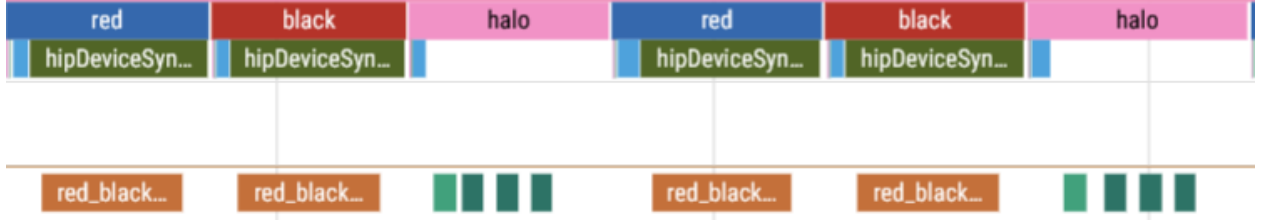


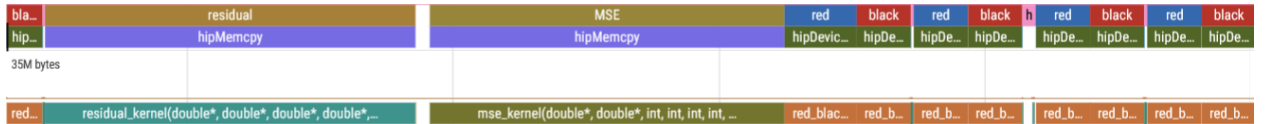
Figure 3: With GPU-aware. For a rank with 3 neighbors, only 3 device-to-device copies are needed.

4.4 Blocking

We experimented with loading u in thread block shared memory during the red-black update to speed up repeated memory accesses from different threads. However, this offered no speedup, which we believe is because the GPU already caching this memory for us.

4.5 GPU Profiling

GPU profiling with rocprofv3 helped reveal the benefits of the GPU-aware MPI communication, and revealed other characteristics of our solver.



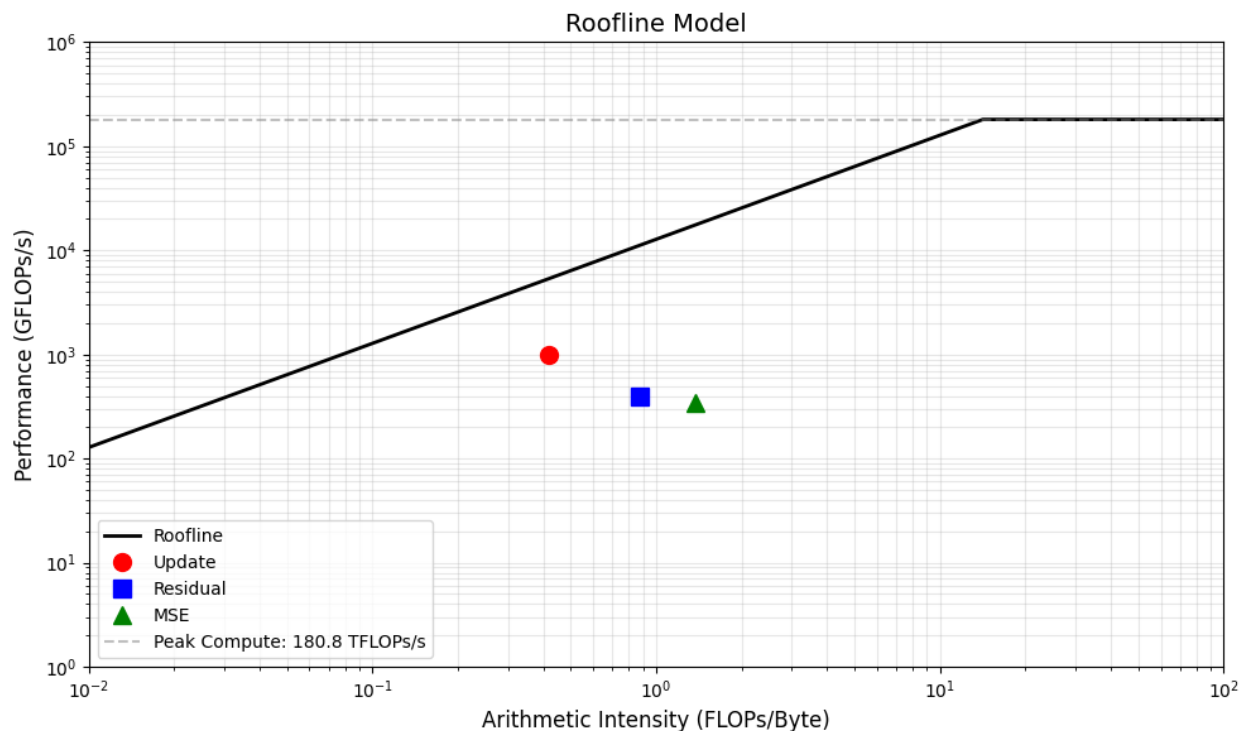
4.6 Performance

	Avg Time (s)	GFLOPs Required	Performance (GFLOP/s)
Update	0.0873326	85.9	983.6
Residual	0.303846	120.3	395.9
MSE	0.273026	94.49	346

Table 1: Measured performance for 2048x2048x2048 grid, 2x2x2 MPI ranks, 8 GPUs

4.7 Roofline

We plot the achieved performance from above with a roofline model.



5 Discussion

In the roofline above, the residual and MSE calculations are more arithmetically intense than the red-black update, but achieve less performance. Their work cannot be fully parallelized, and require communication to reduce across all MPI ranks. The red-black update gets the closest to the theoretical performance bounds from the roofline. MPI communication is likely what prevents us from getting closer. One thing we considered is that we may want to avoid pre-computing f on the grid, which moves the red-black update further to the right on the roofline, increasing the possible peak performance. But practically, one may not know the function f , only its values on the grid, or it may be very expensive to compute.

There still optimizations left unexplored. Better solvers, e.g. conjugate gradient, could give faster convergence. Using shuffles could let us compute reductions for the residual and MSE GPU kernels much faster. There may be more optimization techniques that we are not familiar with.

6 GenAI Acknowledgment

Claude was used on occasion to help write code and debug, and also to generate plots.